



I. Introduction to Docker

What is Docker?

Docker is a powerful platform that allows developers and system administrators to build, run, and share applications packaged in **containers**.

What is a Container?

- A container is like a lightweight virtual machine.
- It includes your app and everything it needs to run (libraries, environment, files).
- It shares the host OS kernel, making it more efficient than a full VM.

Imagine a container as a small, self-contained box that includes everything your application needs to work: the code, libraries, configuration files, dependencies, and system tools. Once built, this container can run anywhere—on your laptop, a test server, a cloud instance—and it will always behave the same way.

This solves one of the oldest problems in software development: **"It works on my machine!"**

Why Docker is Useful:

- **Portability:** Containers can run consistently across different environments.
- **Speed:** Containers start almost instantly and use fewer system resources than virtual machines.
- **Isolation:** Each container runs in its own environment, preventing conflicts between apps.
- **Scalability:** You can run multiple containers at once to handle increased load.

II. Core Concepts of Docker

Key Terms

1. Images: An image is a read-only template used to create a Docker container. Think of it like a recipe: it describes the operating system, the software installed, and the app itself.

2. Containers: A container is a live, running instance of an image. It behaves like a mini-computer running your application, but it's isolated from your system and other containers.

3. Dockerfile: This is a text file with step-by-step instructions on how to build a Docker image. It's like a blueprint for your container.

4. Docker Hub: A public repository for sharing images. Similar to GitHub but for Docker images. You can pull existing images or push your own.

5. Docker Engine: The core part of Docker that runs containers and builds images. It communicates with your operating system to manage resources.

III. Creating a Dockerfile (The Blueprint)

A. Purpose

Defines how to build a Docker image. Think of it like a recipe for making your environment.

- Base image (e.g., Ubuntu, Python)
- Software to install
- Files to include
- Commands to run when the container starts

B. Common Instructions

- `FROM ubuntu:latest`: Start from the latest Ubuntu image.
- `RUN apt-get update && apt-get install`: Run commands during build.
- `COPY . /app`: Copy your project files into the image.
- `CMD ["python3", "app.py"]`: Run this command when the container starts.

C. Real Example

```
FROM ubuntu:latest
RUN apt-get update && apt-get upgrade -y
CMD echo "Hello, Docker World!"
```

This container, when run, simply prints a message.

IV. Building a Web API with Flask and Docker

A. Why Flask?

Flask is:

- Lightweight
- Easy to use
- Perfect for building small APIs

B. Sample Code

```
from flask import Flask
app = Flask(__name__)

@app.route("/api/hello")
def hello():
    return {"message": "Hello, World!"}

app.run(host="0.0.0.0", port=5252)
```

Note: 0.0.0.0 means it's accessible from outside the container.

C. Dockerizing It

Dockerfile:

```
FROM python:3.8-slim
WORKDIR /app
COPY . /app
RUN pip install flask
CMD ["python", "app.py"]
```

Build and Run:

```
docker build -t flask-api .
```

Test in browser: <http://localhost:5252/api/hello>

This image installs Flask and runs your API.

V. Serving a Front-End with Nginx

A. What is Nginx?

Nginx is a fast, lightweight web server used to:

- Serve static content (HTML, CSS, JS)
- Act as a reverse proxy

B. Use Case

You can use it to serve your website's front-end from a container.

C. Dockerfile Example

```
FROM nginx:alpine
COPY ./static-site/ /usr/share/nginx/html
```

Place your HTML files in `static-site/` and they will be shown in the browser.

☐ VI. Connecting Containers: Networking Basics

A. Why Connect Them?

Your front-end needs to talk to your API.

B. Docker Networking

- By default, containers are isolated.

- You can create a **user-defined network** that allows containers to communicate by name.
 - Docker Compose simplifies this greatly.
-

VII. Load Balancing with a Reverse Proxy

A. What is a Reverse Proxy?

A reverse proxy sits in front of your application servers and directs traffic. It's a server that receives client requests and forwards them to other backend servers.

B. Why Use One?

- Centralized traffic control
- Can balance load across multiple containers
- Helps with security and routing

C. Round Robin Load Balancing

What is Round Robin load balancing?

Round Robin is the simplest and most common load balancing algorithm. Round Robin distributes incoming network traffic sequentially across a group of servers in a cyclical, rotating order. Think of it like dealing cards around a table - each server gets a request in turn.

When requests come in:

1. **First request** → sent to **api1:5252**
2. **Second request** → sent to **api2:5252**
3. **Third request** → back to **api1:5252**
4. **Fourth request** → back to **api2:5252**

D. Nginx Config Example (default.conf)

```
http {  
    upstream backend {  
        server api1:5252;  
        server api2:5252;  
    }  
}
```

```
server {  
    listen 80;  
  
    location /api/ {  
        proxy_pass http://backend;  
    }  
  
    location / {  
        proxy_pass http://frontend;  
    }  
}
```

This balances between two Flask APIs and serves the front-end.



VIII. Docker Compose: The Glue

A. What is Docker Compose?

A tool to define and run multi-container Docker applications using [docker-compose.yml](#).

B. Why Use It?

- Automates networking
- Simplifies multi-container setup
- Brings all services together (API, front-end, proxy)

C. Sample File

```
version: '3'  
services:  
  api1:  
    build: ./api  
  api2:  
    build: ./api  
  frontend:  
    build: ./frontend
```

```
proxy:
  build: ./proxy
  ports:
    - "80:80"
```

Key Docker Compose Commands

1. Build the images for all services

`docker compose build`

- Reads the `docker-compose.yml` file and builds all listed images.

2. Start all containers defined in the file

`docker compose up`

- Starts and runs all the services.
- Add `-d` to run in the background:

`docker-compose up -d`

3. Stop and remove all containers

`docker compose down`

- Stops running containers and removes them, including networks.

These commands allow you to easily spin up and tear down full application stacks.

Bonus Tips:

✓ To List All Docker Images:

Run:

```
docker images
```

This will show something like:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-cow	latest	123abc456def	10 minutes ago	150MB
ubuntu	latest	456def789ghi	2 days ago	77MB

That `hello-cow` image is the one you built. If you forget the name, you'll still see it here.

Clean Up Unused Images

If you want to delete it later:

```
docker rmi hello-cow
```

Or if you just want to delete **dangling/unused** images:

```
docker image prune
```

Want to See All Containers You Ran?

```
docker ps -a
```

This shows all containers (even stopped ones), so you can track when and what you ran.

✓ Step-by-Step: Launch Ubuntu Container

Run this command:

```
sudo docker run -it ubuntu
```

🧠 What This Does:

- `docker run`: starts a container
 - `-i`: keeps STDIN open (interactive)
 - `-t`: allocates a pseudo-terminal (so it feels like a shell)
 - `ubuntu`: tells Docker which image to use
-

💻 Once Inside the Container

You'll see a prompt like:

```
root@container_id:/#
```

Try these commands to test it out:

```
ls          # list files (you'll see it's almost empty!)  
pwd         # print current directory  
uname -a    # check Linux kernel info  
apt update  # update package index
```

When You're Done

To exit the container:

```
exit
```